

Assignment 1 by Chris

```
# Define a list of your top 3 favorite hobbies
favorite_hobbies = ["Reading", "Photography", "Hiking"]

# Print the list
print("My top 3 favorite hobbies are:")
print(favorite_hobbies)

# Print each hobby with numbering
print("\nMy top 3 favorite hobbies (numbered):")
for i, hobby in enumerate(favorite_hobbies, start=1):
    print(f"{i}. {hobby}")
```

```
My top 3 favorite hobbies are:
['Reading', 'Photography', 'Hiking']
```

```
My top 3 favorite hobbies (numbered):
1. Reading
2. Photography
3. Hiking
```

Data Manipulation with Pandas

```
import pandas as pd

# Create a sample dataset using a dictionary
data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'Diana', 'Eva'],
    'Age': [25, 30, 35, 28, 22],
    'City': ['New York', 'London', 'Paris', 'Tokyo', 'Sydney'],
    'Occupation': ['Engineer', 'Doctor', 'Teacher', 'Designer', 'Student'],
    'Salary': [75000, 120000, 65000, 80000, 30000]
}

# Convert dictionary to Pandas DataFrame
df = pd.DataFrame(data)

# Display the DataFrame
print("Sample Dataset:")
print(df)

# Display additional information about the DataFrame
print("\n" + "="*50)
print("\nDataFrame Info:")
print(df.info())

print("\n" + "="*50)
```

```
print("\nStatistical Summary:")
print(df.describe())
```

Sample Dataset:

	Name	Age	City	Occupation	Salary
0	Alice	25	New York	Engineer	75000
1	Bob	30	London	Doctor	120000
2	Charlie	35	Paris	Teacher	65000
3	Diana	28	Tokyo	Designer	80000
4	Eva	22	Sydney	Student	30000

=====

DataFrame Info:

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 5 entries, 0 to 4
```

```
Data columns (total 5 columns):
```

#	Column	Non-Null Count	Dtype
0	Name	5 non-null	object
1	Age	5 non-null	int64
2	City	5 non-null	object
3	Occupation	5 non-null	object
4	Salary	5 non-null	int64

```
dtypes: int64(2), object(3)
```

```
memory usage: 332.0+ bytes
```

```
None
```

=====

Statistical Summary:

	Age	Salary
count	5.000000	5.000000
mean	28.000000	74000.000000
std	4.949747	32287.768582
min	22.000000	30000.000000
25%	25.000000	65000.000000
50%	28.000000	75000.000000
75%	30.000000	80000.000000
max	35.000000	120000.000000

Data Visualization with Matplotlib/Seaborn

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np

# Create the sample dataset
data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'Diana', 'Eva'],
    'Age': [25, 30, 35, 28, 22],
    'City': ['New York', 'London', 'Paris', 'Tokyo', 'Sydney'],
    'Occupation': ['Engineer', 'Doctor', 'Teacher', 'Designer', 'Student'],

```

```

    'Salary': [75000, 120000, 65000, 80000, 30000]
}

df = pd.DataFrame(data)

# Set up the plotting style
plt.style.use('seaborn-v0_8-darkgrid')
fig, axes = plt.subplots(2, 2, figsize=(14, 10))
fig.suptitle('Employee Data Visualizations', fontsize=16, fontweight='bold')

# 1. BAR CHART: Salaries by Employee
axes[0, 0].bar(df['Name'], df['Salary'], color='skyblue', edgecolor='navy', lir
axes[0, 0].set_title('Salaries by Employee', fontsize=12, fontweight='bold')
axes[0, 0].set_xlabel('Employee Name')
axes[0, 0].set_ylabel('Salary ($)')
axes[0, 0].tick_params(axis='x', rotation=45)
# Add value labels on bars
for i, v in enumerate(df['Salary']):
    axes[0, 0].text(i, v + 2000, f'${v:,}', ha='center', fontsize=9)

# 2. BAR CHART: Age Distribution
axes[0, 1].bar(df['Name'], df['Age'], color='lightcoral', edgecolor='darkred',
axes[0, 1].set_title('Age Distribution by Employee', fontsize=12, fontweight='t
axes[0, 1].set_xlabel('Employee Name')
axes[0, 1].set_ylabel('Age (years)')
axes[0, 1].tick_params(axis='x', rotation=45)
# Add value labels
for i, v in enumerate(df['Age']):
    axes[0, 1].text(i, v + 0.5, str(v), ha='center', fontsize=9)

# 3. LINE PLOT: Salary Trend (sorted by Age)
# Sort by Age to show trend
df_sorted = df.sort_values('Age')
axes[1, 0].plot(df_sorted['Age'], df_sorted['Salary'],
                marker='o', linewidth=2, markersize=8, color='green',
                linestyle='-', markerfacecolor='darkgreen')
axes[1, 0].set_title('Salary vs Age (Line Plot)', fontsize=12, fontweight='bolc
axes[1, 0].set_xlabel('Age (years)')
axes[1, 0].set_ylabel('Salary ($)')
axes[1, 0].grid(True, alpha=0.3)
# Add labels for each point
for i, row in df_sorted.iterrows():
    axes[1, 0].annotate(f'{row["Name"]}\n${row["Salary"]:,}',
                        xy=(row['Age'], row['Salary']),
                        xytext=(5, 5), textcoords='offset points', fontsize=8)

# 4. LINE PLOT with Multiple Lines: Salary and Age trends
# Create x-axis as index
x = np.arange(len(df))
axes[1, 1].plot(x, df['Salary'], marker='s', linewidth=2, markersize=8,
                label='Salary', color='blue')

```

```

axes[1, 1].plot(x, df['Age'] * 2000, marker='o', linewidth=2, markersize=8,
                label='Age (scaled by 2000)', color='orange') # Scale Age for
axes[1, 1].set_title('Salary and Age Comparison', fontsize=12, fontweight='bold')
axes[1, 1].set_xlabel('Employee Index')
axes[1, 1].set_ylabel('Salary ($)')
axes[1, 1].set_xticks(x)
axes[1, 1].set_xticklabels(df['Name'], rotation=45)
axes[1, 1].legend()
axes[1, 1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

```



Advanced Features

✓ Load Data from Google Drive

```

import pandas as pd

# Specify the path to your CSV file on Google Drive
csv_file_path = '/content/drive/MyDrive/willisCollege/datasett.csv' # Replace w

# Load the CSV file into a pandas DataFrame
try:
    df_from_drive = pd.read_csv(csv_file_path)
    print(f"Successfully loaded '{csv_file_path}'")
    # Display the first 5 rows of the DataFrame
    print("\nFirst 5 rows of the DataFrame:")
    display(df_from_drive.head())

```

```
except FileNotFoundError:
    print(f"Error: The file '{csv_file_path}' was not found. Please check the p
except Exception as e:
    print(f"An error occurred while loading the CSV: {e}")
```

Successfully loaded '/content/drive/MyDrive/willisCollege/datasett.csv'

First 5 rows of the DataFrame:

	text 
0	I love this movie! It's fantastic and well-acted.
1	This product is terrible. I would not recommen...
2	The service was okay, but the food was amazing!
3	I am so disappointed with this purchase. Waste...
4	Absolutely wonderful experience. Would buy again!

```
# Another example of measuring execution time of a list comp
%timeit [x + 1 for x in range(10000) if x % 2 == 0]
```

708 μ s $\pm$ 9.3 μ s per loop (mean $\pm$ std. dev. of 7 runs, 1000 loops each)

In this exercise, i learned how to perform several key operations:

Mount Google Drive: We successfully mounted Google Drive to access files stored in the cloud directly from the Colab environment. Load CSV from Drive: We loaded a CSV file from Google Drive into a pandas DataFrame, demonstrating how to handle potential FileNotFoundError. Display DataFrame Information: We displayed the head of the DataFrame to get a quick overview of the loaded data. Measure Code Execution Time: We utilized the %timeit magic command to efficiently measure the execution time of Python code, specifically for list comprehensions, which is useful for performance optimization.